CS 142: Program design and methodology
Spring Term, 2025

Lab 2

In this lab, you'll practice writing test cases.

Testing PrintQueue

Begin by loading the project with the code. You can download the given code from the course webpage (`http://courses.knox.edu/cs142` under assignments) or from Google Classroom. Unzip it and open the directory it contains. Then double click on the file `lab2.code-workspace`, which is a workspace in VSCode. This will open the project. Select the `TestPrintQueue.java` file in the `src` subdirectory. The workspace all enables JUnit test cases.

The project also contains an interface `PrintQueue` that represents the system to which you submit print jobs. For our purposes, we will not use real print jobs, but will instead just use String which represents the name of a file to print. The interface has the following methods:

- A method `submit` that takes the file name as a string and a boolean indicating whether the job comes from a manager or not. (Note that a real print queue would also take the file itself instead of a file name, but we're doing it this way for simplicity.) This is the method called by users who submit jobs.
- A method `hasNextJob` that takes no arguments and returns a boolean indicating whether there is a job currently in the queue (i.e. one that has been submitted, but not yet removed).
- A method `nextJob` that takes no arguments, but returns the file name of the next job to run (and removes it from the queue). Jobs submitted by managers will be returned first, with non-manager jobs only being returned when there are no manager-submitted jobs waiting. Within the class of manager jobs, the jobs will be returned in the order in which they were submitted. Similarly, within the class of non-manager jobs, jobs will be returned in the order in which they were submitted. If this method is called when there are no jobs waiting in the queue, it throws the exception `NoSuchElementException`.

The first of these is a method for the user; it's what would be called when you want to print something. The other two are for use by the printer. Whenever the printer is free, it calls `hasNextJob` to check if there is a print job waiting in the `PrintQueue`. If there is, the printer calls `nextJob` to get it for printing.

In addition to the interface itself, the project contains two implementations of it. The implementations are called `WorkingPrintQueue` and `BadPrintQueue`. (Note that you don't have to look at the code for either implementation, which use some features of Java that we haven't covered yet.) In addition to the methods above, each implementation has a constructor that takes no arguments and creates an empty queue. Open the class `TestPrintQueue`, which is where we'll put our test cases. Its test fixture has a single variable `p` of type `PrintQueue`. By changing which line in `setup` is commented out, you can make `p` have either a `BadPrintQueue` or a `WorkingPrintQueue`.

To run a test case, click the little symbol in the left margin by its header. To run them all, click the similar symbol in the left margin of the class header (`public class` etc).

Write test cases for `PrintQueue`, starting by filling in (and renaming) `test1`.. Then use them to find the bug(s) in `BadPrintQueue`; these are indicated by test cases that fail for `BadPrintQueue`, but pass when you switch to `WorkingPrintQueue`. I suggest that you begin by writing some comments about test cases that you'd like to create. What promises does the description of the `PrintQueue` class make?